

Cheat Sheet: Build Self-Improving Agents with LangGraph

Estimated time needed: 10 minutes

Introduction

Modern agent architectures enable AI systems to critique and refine their own output for higher quality. These "self-improving" agents use loops where the agent reviews its work and acts on feedback. LangGraph – a graph-based framework for stateful LLM applications – makes it easy to implement these patterns.

At a high level, these can be categorized as three approaches: Reflection agents, reflection agents, and ReAct agents. Each uses a different strategy for self-improvement:

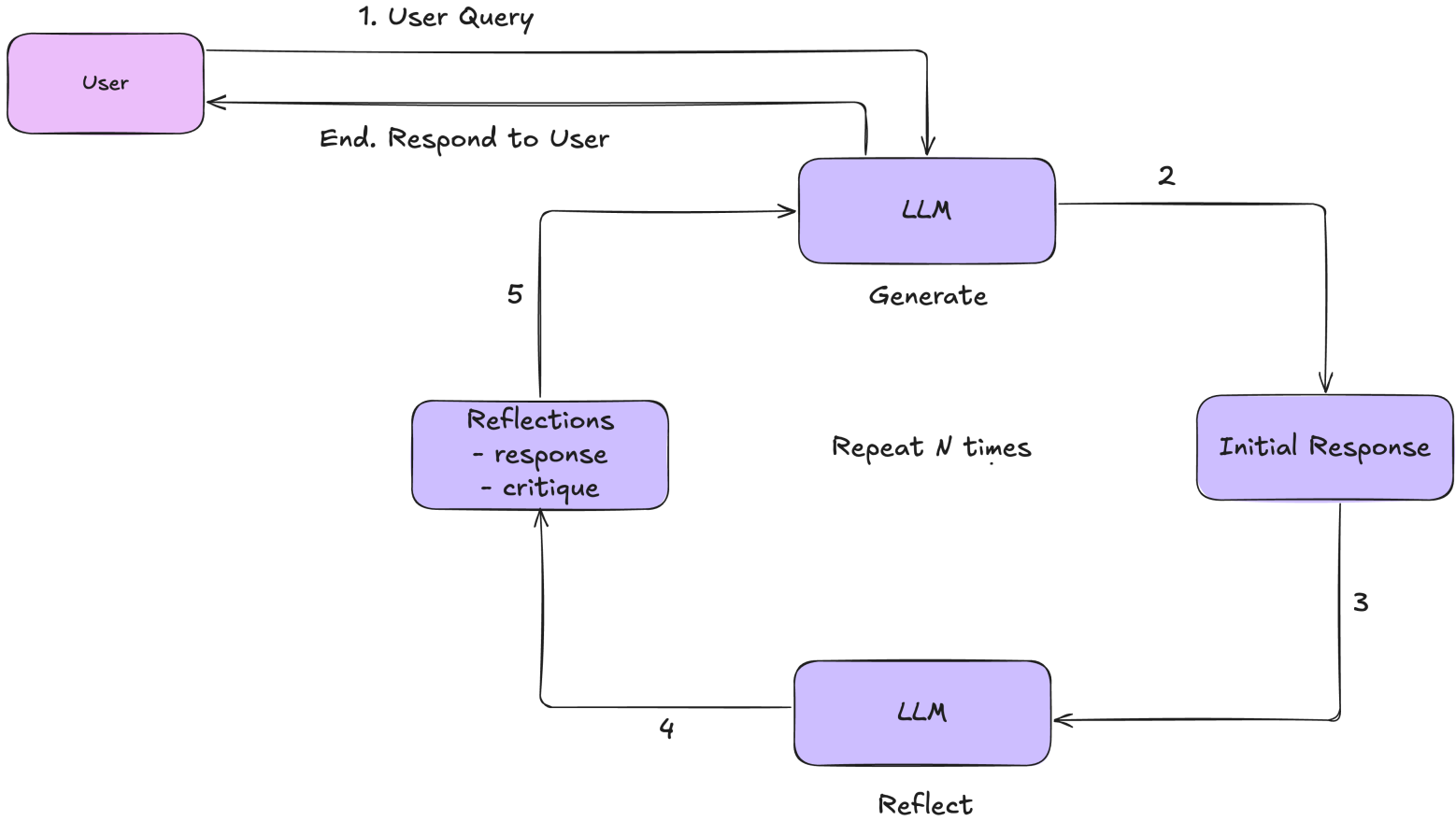
Agent	Description
Reflection agents	Attempts to model its review (its own answer) (like a teacher grading its work)
Reflection agents	Adits external feedback (search or tools) to guide corrections
ReAct agents	Alternates reasoning and actions, thinking and doing in a loop (tool calls, chains-of-thought)

LangGraph represents agents as graphs of states and nodes. The state (often a message history) flows through nodes (functions or LLM calls) linked by edges with conditional logic. Below, we explain each agent type, show sample LangGraph code, and give guidance on success.

Reflection agents

Reflection agents use **internal critique** to refine output. Conceptually, the agent first **generates** an initial answer, then a second step **reflects** on that answer. The reflector (often role played as a teacher or critic) points out flaws or suggests improvements. The agent may loop this generate-then-reflect cycle a few times to polish the answer.

Workflow of a reflection agent:



Concept	Description
Mechanics	Typically, one node calls the LLM to produce a response, and another node calls the LLM to critique or improve it. A simple LangGraph MessageGraph can model this two-step loop.
Example Code	Below, <code>generate_answer</code> and <code>critique_answer</code> are two nodes. We loop between them until a max step count is reached. See the pseudocode below.

Note: The code provided below is pseudocode for learning purposes. If you try to copy paste the below code to an IDE, it won't work.

```
"""
from langgraph.graph import MessageGraph, END
from langchain_core.messages import HumanMessage, AIMessage
# Step that generates an initial response
def generate_answer(state):
    # (In practice, call an LLM here)
    answer = "This is my first attempt."
    return {"messages": state["messages"] + [AIMessage(content=answer)]}
# Node that critiques and refines the previous answer
def critique_answer(state):
    # (In practice, call LLM to critique)
    critique = "The answer is thoughtful, but more detail."
    return {"messages": state["messages"] + [AIMessage(content=critique)]}
# Main loop
builder = MessageGraph()
builder.add_node("generate", generate_answer)
builder.add_node("reflect", critique_answer)
builder.set_entry_point("generate")
# Use control, otherwise will be iterations
MAX_ITER = 5
while True:
    state = builder.write(state)
    builder.add_conditional_edges("reflect", lambda: "generate")
    builder.add_edge("reflect", "generate")
    graph = builder.compile()
    # Run the reflection agent
    initial_message = HumanMessage(content="Explain photosynthesis.")
    result = graph.invoke({"messages": [initial_message]})
    print(result["messages"][-1]) # final answer or critique

```

This makes the agent **self-critique** its answer. In practice, the reflector node is prompted to evaluate the generator's output and suggest corrections. The loop continues until no more revisions are needed or a limit is reached.

• **When to use** Reflection is useful for creative or open-ended tasks (e.g., drafting text, answering complex questions) where iterative refinement helps. It adds overhead (extra LLM calls) but often yields cleaner, more thorough answers. However, since it only relies on the model's own reasoning (no outside data), the final answer may not improve much unless the reflector catches errors. Use Reflection when you want basic iterative self-improvement without adding external searches or tools.

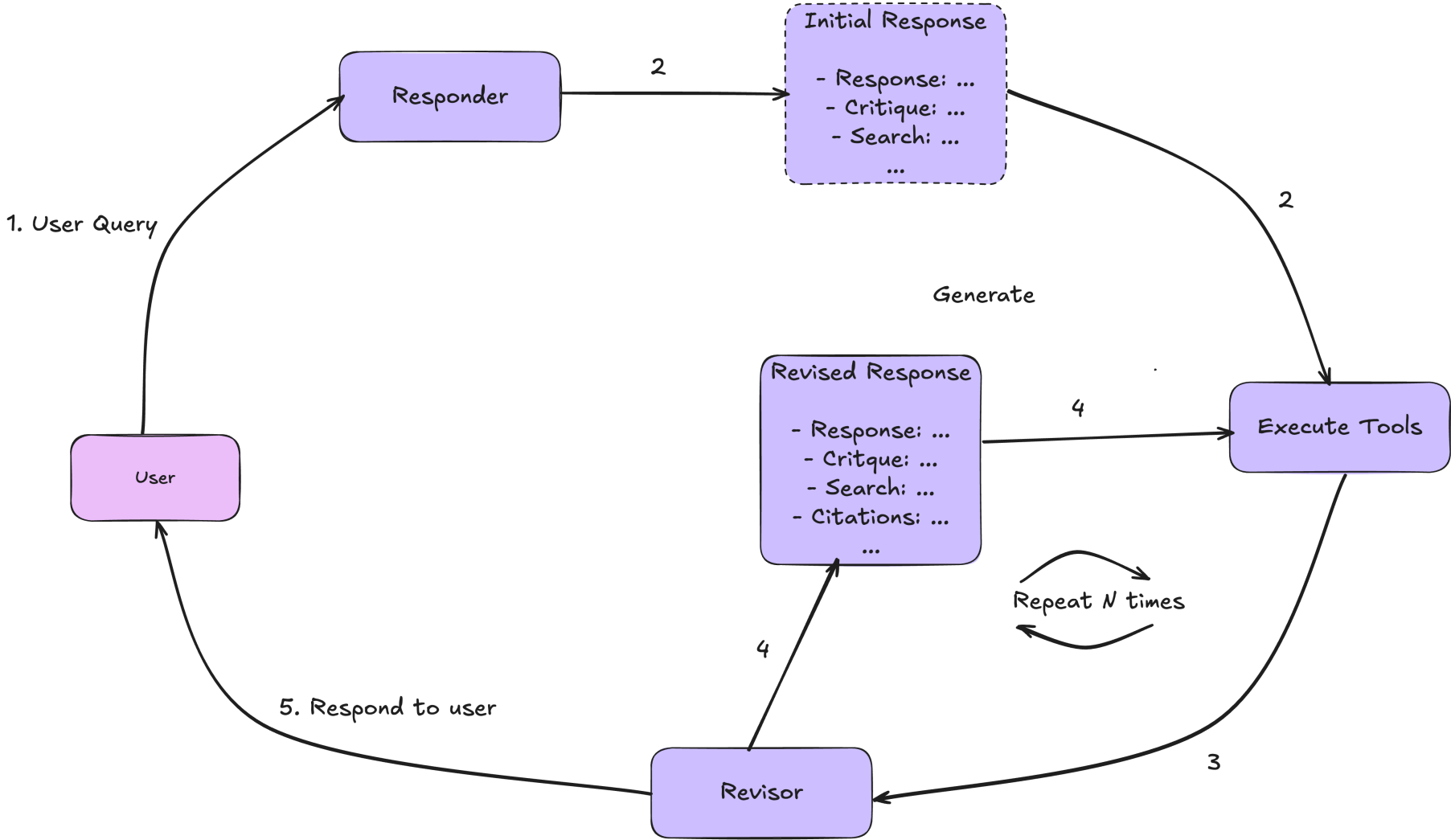
Reflection agents

Reflection agents formalize the idea of reflection with external grounding. Here the agent not only critiques its output, but also uses external information or citations to do so. Each cycle typically involves three steps:

Step	Description
Goal (initial response)	The agent generates an answer and may propose search queries (or tool calls) to gather facts.
Execute tools	These queries are run (for example, web search) and results are added to the context.
Revise	A "revise" node has the agent analyze the draft answer plus fetched info, and explicitly fix missing or incorrect parts.

Reflection forces the agent to cite sources and announce what's missing, making corrections more effective. In LangGraph, we chain these nodes in a loop (Draft → Execute Tools → Revise) until no further revisions are needed or a maximum iteration.

Workflow of a reflexion agent



Concept	Description
Mechanism	Each iteration adds more grounding. For example, after the draft answer, the agent might search Wikipedia, then the revise step reads the search results and updates the answer. The revised answer goes back into the loop if needed.
Workflow rule	Below is a pseudocode of a Reflection-style loop. (tool_search is a stand-in for any external lookup.)

```
graph TD
    User[User] -- "1. User Query" --> Responder[Responder]
    Responder -- "2" --> InitialResponse[Initial Response<br/>- Response: ...<br/>- Critique: ...<br/>- Search: ...<br/>...]
    InitialResponse -- "2" --> ExecuteTools[Execute Tools]
    ExecuteTools -- "3" --> Revisor[Revisor]
    Revisor -- "4" --> RevisedResponse[Revised Response<br/>- Response: ...<br/>- Critique: ...<br/>- Search: ...<br/>- Citations: ...<br/>...]
    RevisedResponse -- "4" --> ExecuteTools
    ExecuteTools -- "3" --> Revisor
    Revisor -- "5. Respond to user" --> User
```

This agent uses a "hall-in-search-or-hall" (generate/analyze) to ground its critique. The revise rule then updates the answer explicitly (e.g., adding evidence or fixing errors). The process stops when the agent judges the answer is good or after a set number of loops.

* "Hall-in-search-or-hall" is used when accuracy or factual grounding matters. Because it references evidence (citations) and points out missing info, it shines on fact-checking, research, or coding tasks where correctness is critical. It is more complex and slower (requires search/tool calls), but yields highly vetted answers. Use Reflection Agents for tasks like data lookup, code generation with static analysis, or any QA requiring references.

ReAct (Reason + Act) agents interleave thinking and action. Rather than a separate “reflector” step, a ReAct agent alternates between internal reasoning (chain-of-thought) and taking actions (tool calls, function calls) in one workflow. Each cycle, the agent decides what to do, does it, then reasons again on the updated state.

```
graph TD; Query((Query)) --> Agent((Agent)); Agent --> ThoughtReasoning((Thought/Reasoning)); ThoughtReasoning --> Action((Action)); Action --> Observation((Observation)); Observation -- "Need More Info" --> ThoughtReasoning
```

[illegible]

Here the agent *thinks* (calls the model) and *acts* (calls a tool) alternately. The `next_step` function checks the content of the last assistant message to decide, in practice, a ReAct agent's prompt would instruct the model to respond either an action or the final answer, and LangGraph routes accordingly.

- When to use ReAct is best for tasks that require tool use or complex planning, like interacting with APIs, databases, or multi-step reasoning. Because it weaves in actions dynamically, it can adapt to tasks (e.g., “Call calculator tool then interpret output”). It is simpler than Reflexion but more powerful than a basic chain-of-thought. Use ReAct agents when you need the model to **reason and perform external actions** in sequence. For quick setups, LangGraph even offers `create_react_agent` to instantiate a standard ReAct pattern with one call.

Comparison of agent styles			
Agent	Reflection agent	Reflection agent	ReAct agent
Core idea	Model critiques its own answer	Model critiques with external feedback and citations	Model reasons and acts (calls tools) in loop
Flowchart	Generate → Reflective → (loop)	Start → Think/Plan → Reason → (loop)	LLM → Reason/Plan/Act → LLM → ...
Graph components	2 nodes (generate, reflect)	3+ nodes (think, create tools, reason)	2 nodes (think, act) with conditional branching
Feedback sources	Internal (LLM self-critique)	External (tool or search results + LLM review)	External (tool calls informed by model reasoning)
Benefits	Simple setup; improves coherence & detail	High accuracy; enhances evidence and completeness	Flexible tool use; handles complex tasks
Drawbacks	May plateau (no new info); extra compute	More complex and slow (searches/tools each loop)	Requires designing tools; complexity in prompts
Use cases	Refining essays, content drafts	Fact-checking, coding, QA with citations	Question answering with APIs, step-by-step tasks

Conclusion

Each architecture adds complexity (and cost in tokens/time) but also power. Reflection is simplest, ReAct adds structure, and Reflection adds grounding. In practice, LangGraph makes it easy to experiment: you can even start with the built-in `create_react_agent` for a ReAct baseline, then customize as needed.

By understanding these patterns, you can build agents that evaluate and refine their own outputs. Whether through introspection or by leveraging tools and external data, self-improving agents aim for higher-quality, more reliable AI behavior.

Author

[Kevin Gurnea](#)

Other Contributor(s)

[Frank Hees](#)



Skills Network

